

Kampanya Öneri Motoru — Teknik Rapor

Proje: CampaignSystem (banch_kampanya_sistemi) **Özellik dalı:** feature/campaign-recommendations · **PR:** #54 **Kapsam:** GET /api/campaign-recommendations uç noktası, arkasındaki skarlama mantığı, SEASONAL_PATTERN referans tablosu ve admin panelindeki "Kampanya Önerileri" ekranı.

1. Amaç

Operatöre, **son dönem kart harcamalarına bakarak hangi merchant kategorisinde kampanya tanımlamaya değer olduğunu** sıralı bir liste hâlinde sunmak. Her satır bir merchant kategorisidir; yanında bir skor, skoru açıklayan gerekçe alanları ve tek tıkla kampanya formunu dolduran bir **taslak** taşır.

Öneriler **hiçbir yerde saklanmaz** — her istekte TRANSACTION tablosu üzerinden yeniden hesaplanır. Kaydedilen tek şey, operatör "Bu öneriyle kampanya oluştur" deyip formu kaydettiğinde CAMPAIGN tablosuna yazılan normal kampanyadır.

Neden heuristik (ML değil)

Skarlama, açık ve deterministik bir formüldür. Ağırlıkları appsettings.json üzerinden ayarlanır; şu an "eğitmek" bu ayarları oynatmak demektir. Skarlama mantığı tek bir metoda (GetSuggestionsAsync içinde) izole edilmiştir; ileride eğitilmiş bir model bu bloğu controller ve ekran hiç değişmeden değiştirebilir (bkz. §11).

2. Dosya haritası

Dosya	Sorumluluk
CampaignSystem/Entities/SeasonalPattern.cs	SEASONAL_PATTERN tablosunun POOCO'su: kategori × ay × ağırlık
CampaignSystem/Data/Configurations/SeasonalPatternConfiguration.cs	EF eşlemesi + HasData seed (takvim öncülleri)
CampaignSystem/Data/Migrations/20260901083535_AddSeasonalPattern.cs	Tabloyu ve seed satırlarını oluşturan migration
CampaignSystem/Configuration/RecommendationOptions.cs	Ayarlar (Recommendation bölümü): pencere uzunlukları, ağırlıklar
CampaignSystem/DTOs/Recommendations/CampaignSuggestionDto.cs	Yanıt tipleri: CampaignSuggestionDto , SuggestionReasonDto , SuggestionDraftDto
CampaignSystem/DTOs/Recommendations/RecommendationQueryDto.cs	İsteğe bağlı sorgu parametreleri
CampaignSystem/Services/Recommendations/ICampaignRecommendationService.cs	Servis sözleşmesi
CampaignSystem/Services/Recommendations/CampaignRecommendationService.cs	Skorlama mantığının tamamı
CampaignSystem/Controllers/CampaignRecommendationsController.cs	GET /api/campaign-recommendations (Admin)
CampaignSystem/Program.cs	DI kaydı + Configure<RecommendationOptions>
CampaignSystem.Tests/CampaignRecommendationServiceTests.cs	6 entegrasyon testi
frontend/src/app/models/recommendation.ts	DTO'ların TypeScript karşılığı
frontend/src/app/services/recommendation.service.ts	getSuggestions() — uç noktayı saran servis
frontend/src/app/campaigns/campaign-suggestions.ts/.html/.css	"Kampanya Önerileri" ekranı
frontend/src/app/campaigns/campaign-form.ts	applySuggestionDraft() — öneriden form ön-dolumu
docs/campaign-recommendations.md	Kısa özet
docs/sample-recommendation-data.sql	Ekranı dolu göstermek için örnek veri

3. Girdi verisi

Motor dört tabloyu okur:

3.1 TRANSACTION

Ana veri kaynağı. İlgili alanlar:

Alan	Kullanım
Amount	Tutar. Alış satırlarında pozitif, iade (refund) satırlarında negatif.
TransactionDate	Pencereye ve "son yarı / önceki yarı" ayırımına girer
MerchantId	MERCHANT üzerinden kategoriye bağlanır; NULL olan satırlar elenir
OriginalTransactionId	Dolu ise satır bir iadedir; alış sayımından çıkarılır

İade satırının tutarı negatif kayıtlı olduğu için **net harcama** = SUM(Amount) — ayrı bir çıkarma gerekmez. (Bu kural RewardCalculator ile aynıdır.)

3.2 MERCHANT / MERCHANT_CATEGORY

`Merchant.MerchantCategoryId` her işlemi bir kategoriye bağlar. Kategoriler `MerchantCategoryConfiguration` içinde sabit id'lerle seed'lidir (1 Gıda/Market ... 20 Kırtasiye/Oyuncak ... 22 Eğlence).

3.3 SEASONAL_PATTERN (bu özellikte geldi)

Takvim önceli: (`MerchantCategoryId`, `Month`, `Weight`). `Weight = 1.00` sıradan bir ay; `> 1` bilinen sezonsal tepe; `< 1` durgunluk. Satırı olmayan (kategori, ay) çifti **1.00** kabul edilir — yalnızca sapan aylar seed'lenmiştir (~106 satır).

Seed değerleri ölçülmüş veri değil, **Türkiye perakende sezonsallığına dayalı öncüllerdir**: okula dönüş Ağustos–Eylül (Kırtasiye, Eğitim, Elektronik), yaz akaryakıt/seyahat (Akaryakıt, Turizm, Havayolları), Kasım elektronik (Efsane Cuma), sezon geçişlerinde giyim (Mart–Nisan, Eylül–Ekim), ilkbahar düğün sezonu (Mobilya, Beyaz Eşya, Kuyum). Faz 2'de bu ağırlıklar gerçek işlem geçmişinden öğrenilebilir.

Örnek seed satırları (`SeasonalPatternConfiguration.BuildSeed`):

```
Kategori 20 (Kırtasiye/Oyuncak): Oca 1.25, Şub 1.20, Nis 1.10, May 0.85, Haz 0.80,
                                Tem 0.90, Ağu 1.55, Eyl 1.60, Eki 0.90, Ara 1.30
Kategori 15 (Eğitim):          Oca 1.20, Şub 1.25, Nis 0.85, May 0.85, Haz 1.10,
                                Tem 1.10, Ağu 1.45, Eyl 1.60, Eki 1.10, Kas 0.90, Ara 0.85
Kategori 7 (Elektronik):       Oca 0.80, Şub 0.80, Mar 0.90, Ağu 1.15, Eyl 1.20,
                                Kas 1.55, Ara 1.25
```

3.4 CAMPAIGN / CAMPAIGN_MERCHANT

"Kapsam boşluğu" sinyali için: bir kategoride **açık veya yaklaşan** (`IsActive = 1` ve `Status <> 'Ended'`) bir kampanya, o kategorideki bir merchant'ı `CAMPAIGN_MERCHANT` ile hedefliyor mu? Hedefliyorsa kategori "kapsanıyor" sayılır. Merchant kriteri hiç olmayan (yatay) kampanya bilerek kapsam sayılmaz (bkz. §9).

4. Ayarlar — RecommendationOptions

`appsettings.json` → `Recommendation` bölümünden bağlanır. Tek istek bazında `RecommendationQueryDto` ile geçici olarak ezilebilir.

Ayar	Varsayılan	Anlamı
<code>LookbackDays</code>	90	Harcama ve trendin okunduğu geçmiş penceresi. Ortadan ikiye bölünüp trend okunur.
<code>HorizonDays</code>	45	Önerilen kampanyanın varsayılan süresi. Hangi ayların sezon ağırlığının ortalanaacağını ve formu dolduran tarihleri belirler.
<code>MinimumSpend</code>	1000	Pencerede bu tutarın altında net harcaması olan kategori elenir.
<code>MaxSuggestions</code>	10	Uç noktanın döndüğü en fazla öneri sayısı.
<code>SpendWeight</code>	1.0	Normalize edilmiş harcama hacminin skordaki ağırlığı.
<code>TrendWeight</code>	1.5	Trendin (son yarı / önceki yarı) skordaki ağırlığı.
<code>SeasonWeight</code>	1.25	Ufuk boyunca beklenen sezonsal artışın skordaki ağırlığı.
<code>CoverageGapBoost</code>	1.75	Hiçbir açık/yaklaşan kampanyanın kapsamadığı kategoriye uygulanan çarpan.
<code>SuggestedRewardRate</code>	0.02	Kategorinin ortalama işlem tutarının, forma önerilen <code>RewardPoint</code> olarak yansıyan oranı.

5. Algoritma — koddan satır satır

Dosya: CampaignSystem/Services/Recommendations/CampaignRecommendationService.cs Metot: GetSuggestionsAsync(RecommendationQueryDto query, CancellationToken)

5.1 Ayarların çözümü ve sınırlandırılması — satır 34–37

```
var lookbackDays = Math.Clamp(query.LookbackDays ?? _options.LookbackDays, 14, 365);
var horizonDays = Math.Clamp(query.HorizonDays ?? _options.HorizonDays, 7, 180);
var minimumSpend = Math.Max(0m, query.MinimumSpend ?? _options.MinimumSpend);
var maxSuggestions = Math.Clamp(query.MaxSuggestions ?? _options.MaxSuggestions, 1, 50);
```

İstek parametresi verilmişse o, yoksa appsettings değeri kullanılır. Math.Clamp ile mantıksız değerler (ör. lookbackDays = 5000) güvenli aralığa çekilir — dışarıdan gelen sorgu string'i uç noktayı bozamaz.

5.2 Zaman pencereleri — satır 39–42

```
var now = DateTime.Now;
var windowStart = now.AddDays(-lookbackDays);
var midPoint = now.AddDays(-lookbackDays / 2.0);
var horizonEnd = now.AddDays(horizonDays);
```

- windowStart ... now → tüm inceleme penceresi (varsayılan son 90 gün).
- midPoint → pencereyi ikiye böler. midPoint ... now **son yarı**, windowStart ... midPoint **önceki yarı**. Trend bu iki yarının karşılaştırmasıdır.
- now ... horizonEnd → önerilen kampanyanın varsayımsal süresi (varsayılan +45 gün). Sezon ağırlığı bu aralığın kapsadığı aylardan hesaplanır.

DateTime.Now (UTC değil) bilinçli — proje genelinde kampanya tarihleri yerel saatle işlenir (CampaignService , batch job).

5.3 Kategori bazında agregasyon — satır 47–61

```
var aggregates = await context.Transactions
    .AsNoTracking()
    .Where(t => t.MerchantId != null
        && t.TransactionDate >= windowStart
        && t.TransactionDate < now)
    .GroupBy(t => t.Merchant!.MerchantCategoryId)
    .Select(g => new CategoryAggregate
    {
        CategoryId = g.Key,
        RecentSpend = g.Sum(x => x.TransactionDate >= midPoint ? x.Amount : 0m),
        PriorSpend = g.Sum(x => x.TransactionDate < midPoint ? x.Amount : 0m),
        PurchaseSpend = g.Sum(x => x.OriginalTransactionId == null ? x.Amount : 0m),
        PurchaseCount = g.Sum(x => x.OriginalTransactionId == null ? 1 : 0)
    })
    .ToListAsync(cancellationToken);
```

Bu tek sorgu SQL Server'a **tek bir** GROUP BY olarak çevrilir. Kategori başına:

- RecentSpend = son yarıdaki Amount toplamı. İade satırları negatif olduğu için koşullu SUM onları da netler.
- PriorSpend = önceki yarıdaki Amount toplamı.
- PurchaseSpend = yalnızca alış satırları (OriginalTransactionId IS NULL) toplamı. Önerilen ödül puanını boyutlamak için (iadelerden arındırılmış "gerçek" ciro).
- PurchaseCount = alış satırı sayısı. SUM(CASE WHEN ... THEN 1 ELSE 0 END) biçimi EF tarafından güvenli çevrilir.

`NetSpend` , `CategoryAggregate` içinde türetilir (satır 262):

```
public decimal NetSpend => RecentSpend + PriorSpend;
```

Pencerede hiç işlem yoksa (`aggregates.Count == 0`) boş liste döner (satır 63–66).

5.4 Yardımcı sözlükler — satır 68–99

Dört ek sorgu, hepsi `AsNoTracking` ve bellek içi sözlüğe çevrilir:

Değişken	Sorgu	Amaç
<code>categoryNames</code>	<code>MERCHANT_CATEGORY → {Id: CategoryName}</code>	Başlık ve DTO için kategori adı
<code>activeMerchantsByCategory</code>	Aktif <code>MERCHANT → {CategoryId: [MerchantId...]}</code>	Taslağın <code>MerchantIds</code> alanı (formda ön-seçili merchant kriteri)
<code>seasonalWeights</code>	<code>SEASONAL_PATTERN</code> (yalnız ufuk ayları) → <code>{{CategoryId, Month}: Weight}</code>	Sezon ağırlığı araması
<code>coveringCampaigns</code>	<code>CAMPAIGN_MERCHANT</code> $\bowtie$ <code>CAMPAIGN</code> (aktif, <code>Status <> Ended</code>) → <code>{CategoryId: [CampaignId...]}</code>	Kapsam boşluğu tespiti

`coveringCampaigns` sorgusu (satır 92–99):

```
var coveringCampaigns = (await context.CampaignMerchants
    .AsNoTracking()
    .Where(cm => cm.Campaign.IsActive && cm.Campaign.Status != CampaignStatus.Ended)
    .Select(cm => new { cm.CampaignId, cm.Merchant.MerchantCategoryId })
    .Distinct()
    .ToListAsync(cancellationToken))
    .GroupBy(x => x.MerchantCategoryId)
    .ToDictionary(g => g.Key, g => g.Select(x => x.CampaignId).OrderBy(id => id).ToList());
```

5.5 Ufuk aylarının hesabı — `MonthsSpanned` , satır 194–207

```
private static List<int> MonthsSpanned(DateTime from, DateTime to)
{
    var months = new List<int>();
    var cursor = new DateTime(from.Year, from.Month, 1);
    while (cursor <= to)
    {
        months.Add(cursor.Month);
        cursor = cursor.AddMonths(1);
    }
    return months.Distinct().ToList();
}
```

`from` 'un ayının 1'inden başlayıp `to` 'yu geçene kadar ay ay ilerler. Örnek: bugün **1 Eylül**, `horizonDays = 45` → `horizonEnd` $\approx$ 16 Ekim → sonuç **[9, 10]**.

5.6 Normalizasyon tabanı — satır 101–106

```
var maxNetSpend = aggregates.Max(a => a.NetSpend);
if (maxNetSpend <= 0m) return [];
```

Tüm kategoriler arasındaki en yüksek net harcama. Her kategorinin harcaması buna bölünerek 0–1 arasına normalize edilir; böylece `SpendWeight` katsayısı hacimden bağımsız anlam taşır.

5.7 Kategori döngüsü — satır 110–179

Her `aggregate` için sırayla:

(a) Eşik filtresi — satır 112–115

```
if (aggregate.NetSpend < minimumSpend) continue;
```

Net harcaması `MinimumSpend` 'in altındaki kategori atlanır.

(b) Kapsam boşluğu + `IncludeCovered` filtresi — satır 117–123

```
var covering = coveringCampaigns.GetValueOrDefault(aggregate.CategoryId, []);
var isCoverageGap = covering.Count == 0;
if (!isCoverageGap && !query.IncludeCovered) continue;
```

Kapsanan kategori, `includeCovered=true` verilmedikçe listeden çıkarılır.

(c) Trend oranı — satır 125–127

```
var trendRatio = aggregate.PriorSpend > 0m
    ? (double)((aggregate.RecentSpend - aggregate.PriorSpend) / aggregate.PriorSpend)
    : (double?)null;
```

$(\text{sonYarı} - \text{öncekiYarı}) / \text{öncekiYarı}$. $0.42 \rightarrow \%42$ artış, $-0.10 \rightarrow \%10$ azalış. Önceki yarı sıfır/negatifse trend hesaplanamaz $\rightarrow$ `null` (skorlamada 0 gibi davranır).

(d) Sezon ağırlığı — satır 129–132

```
var seasonalWeight = horizonMonths
    .Select(month => seasonalWeights.GetValueOrDefault((aggregate.CategoryId, month), 1.0))
    .DefaultIfEmpty(1.0)
    .Average();
```

Ufuk aylarının her biri için `SEASONAL_PATTERN` ağırlığı (yoksa 1.0), sonra ortalama. Örnek — Kırtasiye, aylar [9, 10]: Eylül 1.60, Ekim 0.90 $\rightarrow (1.60 + 0.90) / 2 = 1.25$.

(e) Normalize harcama ve sınırlanmış trend — satır 134–135

```
var normalisedSpend = (double)(aggregate.NetSpend / maxNetSpend); // 0 ... 1
var clampedTrend = Math.Clamp(trendRatio ?? 0.0, -1.0, 3.0); // -100% ... +300%
```

Trend +300% ile sınırlanır ki tek bir uçuk kategori skoru domine etmesin.

(f) Skor formülü — satır 137–143

```

var rawScore =
    _options.SpendWeight * normalisedSpend
    + _options.TrendWeight * clampedTrend
    + _options.SeasonWeight * (seasonalWeight - 1.0);

var score = Math.Max(rawScore, 0.01)
    * (isCoverageGap ? _options.CoverageGapBoost : 1.0);

```

Üç terim:

Terim	Açılım	Anlam
Harcama	$1.0 \times \text{normalisedSpend}$	En yoğun kategori 1.0 puan, yarısı kadar harcayan 0.5
Trend	$1.5 \times \text{clamp}(\text{trend}, -1, 3)$	+%100 büyüyen kategori +1.5; %50 küçülen -0.75
Sezon	$1.25 \times (\text{seasonalWeight} - 1)$	Ağırlık 1.4 ise +0.5; 0.8 ise -0.25; 1.0 ise 0

$\text{Math.Max}(\text{rawScore}, 0.01)$: üç terim birden negatife giderse skoru küçük pozitifte tutar — bu kategoriler listenin dibinde toplanır, sıralama bozulmaz. CoverageGapBoost (1.75): kapsanmayan kategori, aynı ham skorlu kapsanan kategorinin önüne geçer — motorun asıl amacı bu boşlukları göstermek.

(g) Önerilen ödül puanı — satır 145–149

```

var averageTicket = aggregate.PurchaseCount > 0
    ? aggregate.PurchaseSpend / aggregate.PurchaseCount
    : 0m;
var suggestedReward = Math.Max(1m,
    Math.Round(averageTicket * (decimal)_options.SuggestedRewardRate, 0));

```

Kategorinin ortalama alış tutarı $\times$ $\text{SuggestedRewardRate}$ (0.02), tam sayıya yuvarlanır, en az 1. Bağlayıcı değil — operatör formda değiştirir.

(h) DTO'nun kurulması — satır 151–178 Her öneri için:

- **Score** (4 haneye yuvarlı)
- **Headline** $\rightarrow$ `BuildHeadline(...)` (bkz. §5.9)
- **Reason** $\rightarrow$ ham ölçümler: `TotalSpend`, `TransactionCount`, `TrendRatio`, `SeasonalWeight`, `SeasonalMonths`, `IsCoverageGap`, `CoveringCampaignIds`
- **Draft** $\rightarrow$ { `Name`: "<kategori> kampanyası", `StartDate`: bugün, `EndDate`: bugün+ufuk, `SuggestedRewardPoint`, `MerchantCategoryId`, `MerchantIds`: kategorideki aktif merchant'lar }

5.8 Sıralama, kesme, rank — satır 181–189

```

var ranked = scored
    .OrderByDescending(s  $\Rightarrow$  s.Score)
    .Take(maxSuggestions)
    .ToList();

for (var i = 0; i < ranked.Count; i++)
    ranked[i].Rank = i + 1;

```

Skora göre azalan sırala, ilk `MaxSuggestions` taneyi al, 1'den başlayarak `Rank` ata.

5.9 Başlık cümlesi — BuildHeadline , satır 209–243

```
var sentence = $"{categoryName} kategorisinde son {lookbackDays} günde {netSpend:N0} ₺ harcama";

if (trendRatio is ≥ 0.15) sentence += $", harcama {Math.Round(trendRatio.Value*100)} arttı";
else if (trendRatio is ≤ -0.15) sentence += $", harcama {Math.Round(Math.Abs(trendRatio.Value)*100)} azaldı";

if (seasonalWeight ≥ 1.1) sentence += ", önümüzdeki dönem sezonsal olarak yüksek";
else if (seasonalWeight ≤ 0.9) sentence += ", önümüzdeki dönem sezonsal olarak düşük";

sentence += isCoverageGap
    ? " - bu kategoride aktif kampanya yok."
    : $" - {coveringCount} aktif/yaklaşan kampanya zaten kapsıyor.";
```

Eşikler (%15 trend, 1.1/0.9 sezon) cümleye yalnız **anlamlı** sinyalleri koyar; küçük dalgalanmalar metne girmez.

6. Sayısal örnek (canlı Docker verisi)

Örnek veri: docs/sample-recommendation-data.sql (~2200 işlem, Kırtasiye ve Eğitim son yarıda bilerek şişirilmiş). Bugün 1 Eylül, varsayılan ayarlar. maxNetSpend = Elektronik'in **2.039.588 ₺**'si.

Kırtasiye / Oyuncak → skor 8.65, sıra #1

Bileşen	Hesap	Değer
NetSpend	269.094 ₺	
normalisedSpend	269.094 / 2.039.588	0.1319
trend	son yarı $\approx$ 4× önceki yarı	+3.0248 → clamp → 3.0
seasonalWeight	(Eyl 1.60 + Eki 0.90) / 2	1.25
Harcama terimi	1.0×0.1319	0.1319
Trend terimi	1.5×3.0	4.5000
Sezon terimi	$1.25 \times (1.25 - 1.0)$	0.3125
rawScore	toplam	4.9444
score	4.9444×1.75 (kapsam boşluğu)	8.65

Elektronik → skor 1.33, sıra #4

Bileşen	Hesap	Değer
NetSpend	2.039.588 ₺ (en yüksek)	
normalisedSpend	2.039.588 / 2.039.588	1.0000
trend	son yarı < önceki yarı (+ iadeler)	-0.2431
seasonalWeight	(Eyl 1.20 + Eki 1.00) / 2	1.10
Harcama terimi	1.0×1.0	1.0000
Trend terimi	$1.5 \times (-0.2431)$	-0.3647
Sezon terimi	$1.25 \times (1.10 - 1.0)$	0.1250
rawScore	toplam	0.7603
score	0.7603×1.75	1.33

Sonuç: Elektronik en yüksek ciroya (7.5 kat) sahip olduğu hâlde, düşen trendi yüzünden Kırtasiye'nin çok altında kalır. Sıralamayı hacim değil, trend + sezon taşır.

Gerçek yanıt (kısaltılmış):

#1 Kırtasiye / Oyuncak	score=8.65	spend=269094	trend=3.0248	season=1.25	gap=True
#2 Eğitim	score=7.89	spend=637271	trend=2.5061	season=1.35	gap=True
#3 Giyim	score=2.59	spend=437930	trend=0.676	season=1.20	gap=True
#4 Elektronik	score=1.33	spend=2039588	trend=-0.2431	season=1.10	gap=True
#5 Restoran / Yeme-İçme	score=0.69	spend=475933	trend=0.1077	season=1.00	gap=True
#6 Akaryakıt	score=0.02	spend=621652	trend=-0.3884	season=1.025	gap=True
#7 Turizm / Seyahat / Otel	score=0.02	spend=987429	trend=-0.4128	season=1.075	gap=True

?includeCovered=true ile **Gıda / Market** de görünür: gap=False , covers=[1, 2005, 3004] — üç açık/yaklaşan kampanya market merchant'larını hedefliyor.

7. HTTP katmanı

CampaignSystem/Controllers/CampaignRecommendationsController.cs

```
[ApiController]
[Authorize(Roles = "Admin")]
[Route("api/campaign-recommendations")]
public class CampaignRecommendationsController(ICampaignRecommendationService recommendationService)
    : ControllerBase
{
    [HttpGet]
    [ProducesResponseType<List<CampaignSuggestionDto>>(StatusCodes.Status200OK)]
    public async Task<ActionResult<List<CampaignSuggestionDto>>> GetAll(
        [FromQuery] RecommendationQueryDto query,
        CancellationToken cancellationToken)
        => Ok(await recommendationService.GetSuggestionsAsync(query, cancellationToken));
}
```

- **Yetki:** yalnız Admin rolü. Token yoksa 401.
- **Sorgu parametreleri** (hepsi opsiyonel): lookbackDays , horizonDays , minimumSpend , maxSuggestions , includeCovered .
- **Yanıt:** 200 OK + List<CampaignSuggestionDto> . Öneri yoksa boş dizi ([]).

8. Frontend akışı

1. recommendation.service.ts — getSuggestions(query) → GET /api/campaign-recommendations (opsiyonel parametreleri HttpParams 'a çevirir).
2. campaign-suggestions.ts/.html/.css — campaigns/suggestions route'u + sol menüde "Kampanya Önerileri". Öneri kartları: sıra rozeti, kategori adı, "Kampanya yok" / "N kampanya kapsıyor" etiketi, başlık cümlesi, gerekçe (harcama / işlem / trend / sezon / skor), ve **"Bu öneriyle kampanya oluştur"** butonu. Bir de "Kapsananları da göster" anahtarı (includeCovered).
3. **Öneriden forma geçiş** — buton router.navigate(['/campaigns/new'], { state: { campaignDraft: suggestion.draft } }) çağırır.
4. campaign-form.ts → applySuggestionDraft() — yeni kampanya modunda history.state.campaignDraft okunur; varsa name , startDate (ISO'dan yyyy-MM-dd 'ye kesilerek), endDate , rewardPoint alanları patchValue ile doldurulur, selectedMerchants signal'ına önerilen merchant id'leri yazılır, "Öneriden dolduruldu — kaydetmeden önce

gözden geçirin." notu gösterilir. /campaigns/new 'e düz gidildiğinde (state yoksa) hiçbir şey değişmez.

9. Tasarım kararları ve sınır durumlar

Durum / karar	Davranış	Gerekçe
Pencerede hiç işlem yok	<code>[]</code>	Ekranda "Şu an öne çıkan bir kategori yok."
<code>maxNetSpend ≤ 0</code> (tümü iade)	<code>[]</code>	Normalize bölmesi tanımsız olurdu
<code>PriorSpend ≤ 0</code>	<code>trendRatio = null</code> , skorlamada 0	Sıfıra bölme yok; "sonsuz büyüme" abartısı yok
Üç terim de negatif	<code>score = max(raw, 0.01) × boost</code>	Zayıf kategoriler dipte toplanır, sıralama bozulmaz
Merchant kriteri olmayan (yatay) kampanya	Kapsam sayılmaz	Motorun amacı <i>kategoriye hedefli</i> kampanya önermek; tek geniş kampanya tüm önerileri susturmamalı
<code>DateTime.Now</code> (UTC değil)	Yerel saat	Proje geneli kampanya tarih işleme konvansiyonu
Repository yerine doğrudan <code>DbContext</code>	Çok tablolu <code>GROUP BY</code>	Not dosyasındaki mimari standart: gruplama gerektiren karmaşık okuma repository işi değil
Skorlama tek metotta izole	—	Faz 2'de eğitilmiş modelle değiştirilebilir; controller/DTO/ekran sabit kalır

10. Testler

`CampaignSystem.Tests/CampaignRecommendationServiceTests.cs` — gerçek SQL Server'a karşı, her test kendi rollback'li transaction'ında.

Test	Kanıtladığı
<code>Suggests_ABusyCategory_NoCampaignCovers</code>	Yoğun harcamalı + kampanyasız kategori #1 sırada, <code>IsCoverageGap = true</code> , taslakta merchant + puan dolu
<code>Excludes_ACategory_AnOpenCampaignAlreadyCovers</code>	Açık kampanyanın kapsadığı kategori varsayılan listede yok; <code>includeCovered=true</code> ile <code>gap=false</code> + <code>CoveringCampaignIds</code> dolu
<code>NetsRefundsOut_OfTheSpendItScores</code>	10.000 alış – 6.000 iade → <code>TotalSpend = 4.000</code> ; iade satırı işlem sayısına girmez
<code>ReportsARisingTrend_InTheReasonAndHeadLine</code>	<code>LookbackDays=40</code> ile son yarıya yığılan harcama → <code>TrendRatio > 0.5</code> , başlıkta "arttı"
<code>RanksASeasonalCategory_AboveANeutralOne_AtEqualSpend</code>	12 aylık 1.6 ağırlıklı kategori, eşit harcamalı nötr kategoriden yüksek skor/sıra
<code>ReturnsEmpty_WhenNoSpendFallsInsideTheWindow</code>	Pencere dışı (200 gün önce) tek işlem → <code>[]</code>

11. Faz 2 — "gerçek eğitme"

Şu anki skorlama saf heuristik. Değiştirilecek tek yer `GetSuggestionsAsync` içindeki **\$5.7(f)** bloğudur (`rawScore` hesabı) — geri kalan her şey (agregasyon, sözlükler, DTO, controller, ekran) aynı kalır.

Olası yönler:

1. **Sezon ağırlıklarını öğrenme.** SEASONAL_PATTERN şu an elle seed'li. Yeterli geçmiş biriktiğinde her kategori için aylık endeks, kendi işlem geçmişinden hesaplanıp tabloya yazılabilir (ör. `weight[kategori, ay] = o ayın ortalama günlük harcaması / yıllık ortalama`).
2. **Gerçek dış veri.** TCMB'nin haftalık sektörel kart harcama serisi çekilip seed güncellenebilir (2026-09-01'de arandı; net aylık indeks tablosu kamuya açık değil, yön doğrulandı).
3. **ML.NET modeli.** Girdi: kategori $\times$ dönem özellikleri (harcama, trend, sezon, geçmiş kampanya performansı, enrollment oranı). Etiket: geçmiş kampanyaların gerçekleşen ödül / katılım sonucu. `rawScore` yerine modelin tahmini konur. Ağırlıklar `RecommendationOptions` 'tan kalkar, model dosyası diske persist edilir.

Hangi yön seçilirse seçilsin, arayüz sözleşmesi (`ICampaignRecommendationService` , `CampaignSuggestionDto`) ve HTTP/ekran katmanı değişmez.

Ek — eşdeğer tek SQL sorgusu (bilgi amaçlı)

Servisin yaptığı işin özü, tek bir sorgu olarak:

```

DECLARE @now datetime2      = SYSDATETIME();
DECLARE @start datetime2    = DATEADD(DAY, -90, @now);
DECLARE @mid datetime2      = DATEADD(DAY, -45, @now);

;WITH agg AS (
    SELECT m.MerchantCategoryId AS CategoryId,
           SUM(CASE WHEN t.TransactionDate ≥ @mid THEN t.Amount ELSE 0 END) AS RecentSpend,
           SUM(CASE WHEN t.TransactionDate < @mid THEN t.Amount ELSE 0 END) AS PriorSpend,
           SUM(CASE WHEN t.OriginalTransactionId IS NULL THEN t.Amount ELSE 0 END) AS PurchaseSpend,
           SUM(CASE WHEN t.OriginalTransactionId IS NULL THEN 1 ELSE 0 END) AS PurchaseCount
    FROM [TRANSACTION] t
    JOIN MERCHANT m ON m.Id = t.MerchantId
    WHERE t.TransactionDate ≥ @start AND t.TransactionDate < @now
    GROUP BY m.MerchantCategoryId
),
season AS ( -- ufuk ayları [9, 10] için ortalama ağırlık
    SELECT MerchantCategoryId, AVG(Weight) AS SeasonalWeight
    FROM SEASONAL_PATTERN WHERE Month IN (9, 10)
    GROUP BY MerchantCategoryId
),
covered AS (
    SELECT DISTINCT m.MerchantCategoryId
    FROM CAMPAIGN_MERCHANT cm
    JOIN CAMPAIGN c ON c.Id = cm.CampaignId
    JOIN MERCHANT m ON m.Id = cm.MerchantId
    WHERE c.IsActive = 1 AND c.Status <> 'Ended'
)
SELECT a.CategoryId,
       mc.CategoryName,
       (a.RecentSpend + a.PriorSpend) AS NetSpend,
       CASE WHEN a.PriorSpend > 0
            THEN (a.RecentSpend - a.PriorSpend) / a.PriorSpend END AS TrendRatio,
       COALESCE(s.SeasonalWeight, 1.0) AS SeasonalWeight,
       CASE WHEN cv.MerchantCategoryId IS NULL THEN 1 ELSE 0 END AS IsCoverageGap
FROM agg a
JOIN MERCHANT_CATEGORY mc ON mc.Id = a.CategoryId
LEFT JOIN season s ON s.MerchantCategoryId = a.CategoryId
LEFT JOIN covered cv ON cv.MerchantCategoryId = a.CategoryId
WHERE (a.RecentSpend + a.PriorSpend) ≥ 1000
ORDER BY NetSpend DESC;

```

Skor karışımı (`normalisedSpend` , `clamp` , ağırlıklar, boost, `Math.Max` tabanı) ve başlık cümlesi kasıtlı olarak uygulama tarafında tutulur — ayarlanabilir kalması ve ileride bir modelle değiştirilebilmesi için.

Hazırlanma: 2026-09-01 · Dal `feature/campaign-recommendations` · Kod referansları `CampaignRecommendationService.cs` (265 satır) satır numaralarına göredir.